

Loom video link

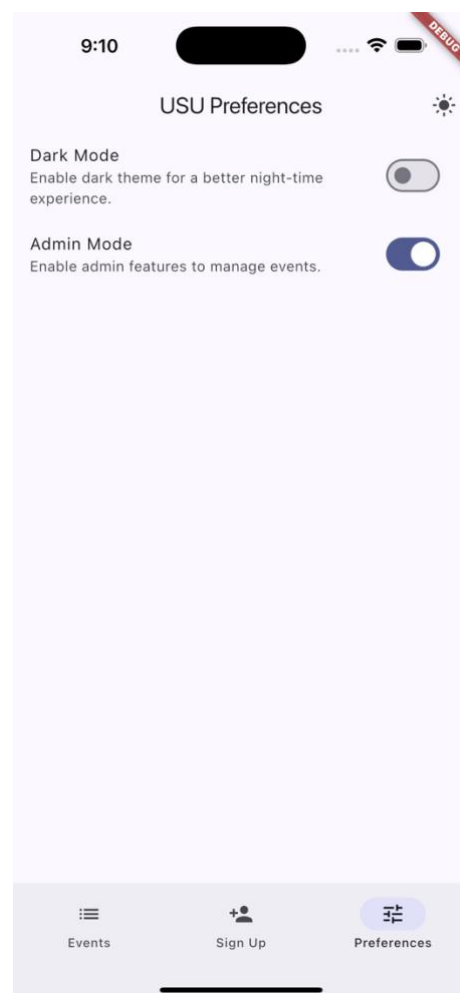
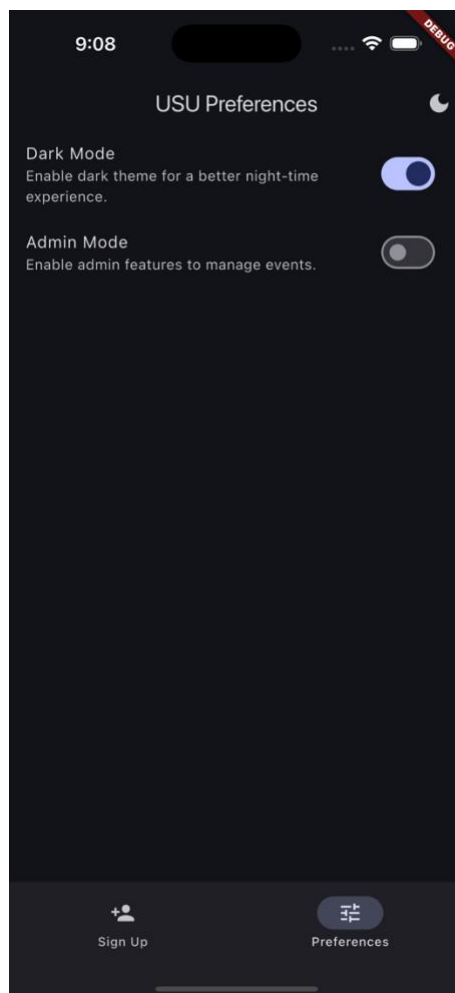
<https://www.loom.com/share/05fc48f18722464ba78bddfdb849bae4?sid=96b1cf42-bffc-48a9-8d52-8379218d5ef9>

Reflection (3–5 sentences)

Notable changes are below. Create an event application for United Stars for the Upstate (USU). USU is a volunteer organization where workers help out with concession to help pay for their children's sport or education activities. This app allows the volunteers to sign up for available events and the administrator can also add upcoming events.

Limitations right now of the app is that it does not track sign on, so the information regarding sign up is currently tied to the event. Future versions of this app could include a separate table `signup` that would hold user along with event id signed up for.

The application uses `shared\_preferences` to toggle two modes (e.g. Dark, Admin). Dark Mode allows the user to toggle the light/dark theme for the application and Admin Mode toggle the Events screen to show allowing and admin to enter/modify events for the organization.



I used SQLite to store the Event instances into a `usu\_events.db` which was managed USUDatabase class. This class had methods to insert/update Event records, retrieve all Event records on initState(), and delete Events per id. The EventModel class was used to instantiate this USUDatabase() and be the main accessor to the database through the UI Widgets.

```

cpsc-6150-solos > solo_04 > lib > services > events_db.dart > USUDatabase > database
1 import 'package:sqflite/sqflite.dart';
2 import 'package:path/path.dart';
3 import 'package:solo_04/models/event.dart';
4
5 /// A small sqflite-backed service to persist 'EventModel' instances.
6 /// This keeps the schema close to the in-memory 'EventModel' so we can
7 /// easily migrate the ValueNotifier-backed repository to persistent storage.
8
9 class USUDatabase {
10   // Singleton pattern so we don't open multiple DB connections accidentally.
11   static final USUDatabase _instance = USUDatabase._internal();
12   factory USUDatabase() => _instance;
13   USUDatabase._internal();
14
15   static const _dbName = 'usu_events.db';
16   static const tableEvents = 'events';
17
18   // Column names
19   static const colId = 'id';
20   // static const colArrivalDate = 'arrival_date';
21   // static const colArrivalTime = 'arrival_time';
22   static const colArrivalDateTimeIso = 'arrival_date_time_iso';
23   static const colLocation = 'location';
24   static const colSize = 'size';
25   static const colShow = 'show';
26   static const colImageAsset = 'image_asset';
27   static const colSignedUp = 'signed_up';
28   static const colAllowSignUp = 'allow_sign_up';
29
30   Database? _db;
31
32   // Lazily open (or return existing) DB instance and create schema if needed.
33   // Version numbers should be incremented when making schema changes.
34   // $colArrivalDate TEXT NOT NULL,
35   // $colArrivalTime TEXT NOT NULL,
36   Future<Database> get database async {
37     if (_db != null) return _db;
38     final path = join(await getDatabasesPath(), _dbName);
39     _db = await openDatabase(
40       path,
41       version: 1,
42       onCreate: (Database db, int version) async {
43         await db.execute('''
44           CREATE TABLE $tableEvents (
45             $colId TEXT PRIMARY KEY,
46             $colArrivalDateTimeIso TEXT NOT NULL,
47             $colLocation TEXT NOT NULL,
48             $colSize TEXT NOT NULL,
49             $colShow TEXT NOT NULL,
50             $colImageAsset TEXT,
51             $colSignedUp INTEGER NOT NULL,
52             $colAllowSignUp INTEGER NOT NULL
53           );
54         ''');
55       },
56     );
57     return _db!;
58   }
59 }

```

```

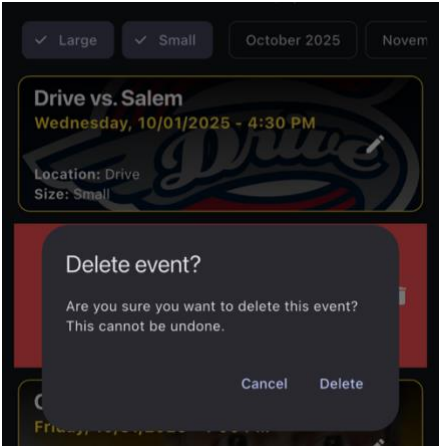
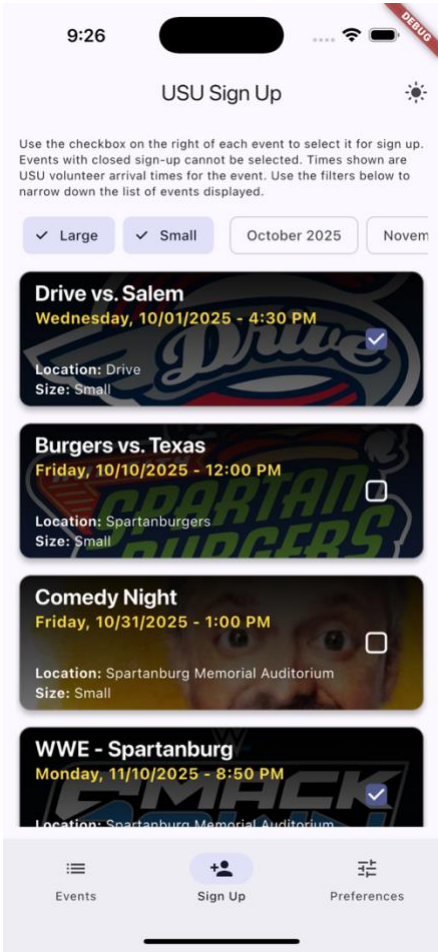
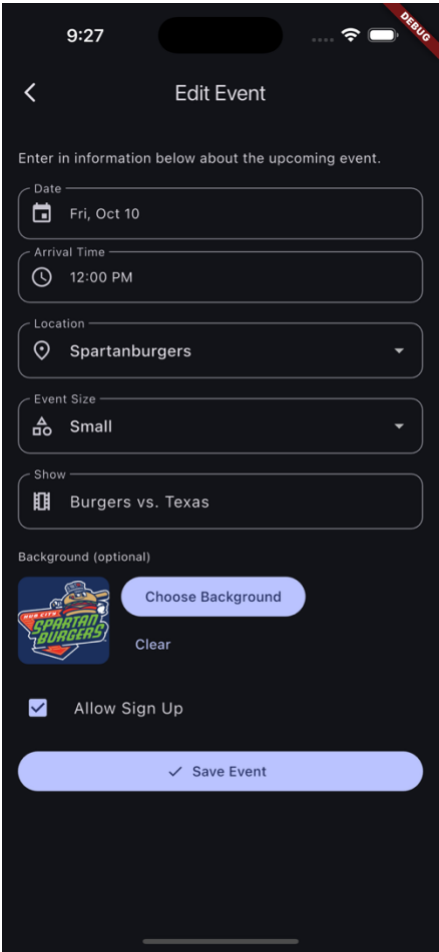
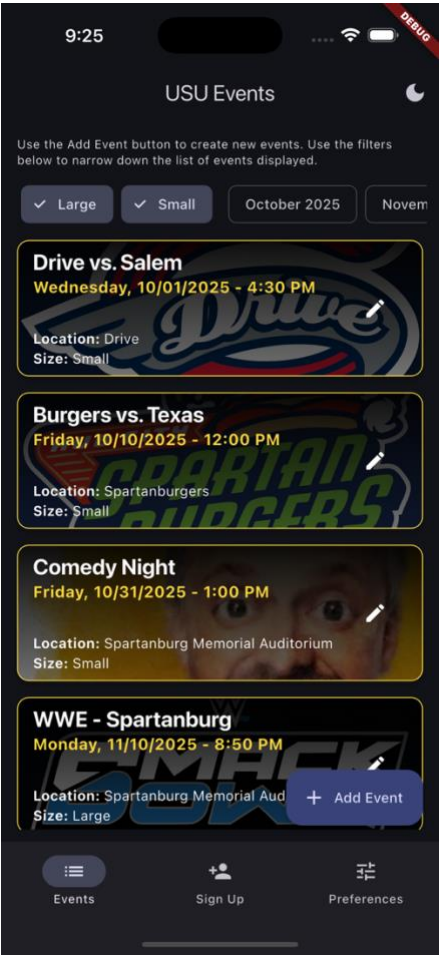
cpsc-6150-solos > solo_04 > lib > models > event_model.dart > EventModel > update
You, 1 hour ago | 1 author (You)
1 import 'package:flutter/foundation.dart';
2 import 'package:solo_04/models/event.dart';
3 import 'package:solo_04/services/events_db.dart';
4
5 You, 1 hour ago | 1 author (You)
6 class EventModel {
7   EventModel._private();
8   static final EventModel instance = EventModel._private();
9
10   final ValueNotifier<List<Event>> events = ValueNotifier<List<Event>>([]);
11
12   final USUDatabase _db = USUDatabase();
13
14   /// Load events from the database into the in-memory list.
15   Future<void> loadEvents() async {
16     final dbEvents = await _db.fetchAllEvents();
17     events.value = dbEvents;
18   }
19
20   void add(Event e) {
21     events.value = [...events.value, e];
22
23     // Persist the new event to the database.
24     _db.upsertEvent(e);
25   }
26
27   void update(Event updated) {
28     events.value = events.value
29       .map(e => e.id == updated.id ? updated : e)
30       .toList();
31
32     // Update the database record as well.
33     _db.upsertEvent(updated);
34   }
35
36   void toggleSignUp(String id, bool value) {
37     final idx = events.value.indexWhere((e) => e.id == id);
38     if (idx < 0) return;
39     final newList = List<Event>.from(events.value);
40     newList[idx] = newList[idx].copyWith(signedUp: value);
41     events.value = newList;
42
43     // Update the database record as well.
44     _db.upsertEvent(newList[idx]);
45   }
46
47   /// Enable or disable sign-up for an event.
48   /// Encapsulates mutations to 'allowSignUp'.
49   void setAllowSignUp(String id, bool value) {
50     final idx = events.value.indexWhere((e) => e.id == id);
51     if (idx < 0) return;
52     final newList = List<Event>.from(events.value);
53     newList[idx] = newList[idx].copyWith(allowSignUp: value);
54     events.value = newList;
55
56     // Remove an event by id.
57     // void remove(Event id) {
58   }
59 }

```

SQLite `events` table was used to house the Events metadata. Each event used a UUID value for uniqueness. Most of the other field values were entered in from the EventForm screen. For the next version of this app, the `signed\_up` field needs to be moved to another table `signups` that will capture user and their corresponding signed up events.

events [usu_events.db]									
WHERE									
ORDER BY									
	id	arrival_date_time_iso	location	size	show	image_asset	signed_up	allow_sign_up	
1	51f68d2f-80ea-4ba8-8f5a-171c4834b5b4	2025-10-31T13:00:00.000	sma	small	Comedy Night	assets/gallery/nate-bargatze.svg	0		1
2	971a9f69-4739-46f7-8085-a487235fd8fc	2025-12-11T11:00:00.000	bswa	large	Harlem Globetrotters	assets/gallery/harlem-globetrotters.svg	0		0
3	84e3182e-4eb3-4793-bc29-b4356ca51690	2025-11-10T20:50:00.000	sma	large	WWE - Spartanburg	assets/gallery/wwe-smackdown.svg	1		1
4	bbcc1fc-c2ab-410f-a9c5-b5e241aa2b6e	2025-10-01T16:30:00.000	drive	small	Drive vs. Salem	assets/gallery/greenville-drive.svg	1		1
5	bd1870d4-f7a1-45bc-8d58-809149e22869	2025-10-10T12:00:00.000	spartanburgers	small	Burgers vs. Texas	assets/gallery/spartanburgers.svg	0		1

Here are the different screens for the application. The Events screen shows only for the Admin Mode and allows the admin to add/modify events. Admins can swipe to delete the event (shown in the bottom image). The Edit Event screen allows the admin to modify details for the event, changing date, arrival time, location, size, show name, and background image. Far right, the Sign Up screen allows the volunteers to check which event they would like to work.



Code Repo

https://github.com/ztraboo/cpsc-6150-solos/tree/assignment-solo-4/solo_04